PRACTICAL OPERATIONS GUIDE

A booking, from setup to completed stay

A step-by-step guide to the current Destination Management System PMS, including property setup, manual bookings, API test bookings, payments, checkout, Night Audit, and the real status of registration roles.

Important: the percentages in this guide are learning milestones, not a progress meter in the PMS. Always verify the saved reservation, invoice, payment, and room status in the application.

Read this first

Area	Current reality
PMS reservations	MongoDB-backed creation, status changes, guest information, room assignment, and audit history.
Invoice and payment	A confirmed reservation creates an issued invoice. Payments can be posted from Financials > Invoices.
Channel Manager	Dashboard previews and channel booking list are not a live provider feed. Full Sync and Pull Future Reservations are local simulations.
OTA booking API	A separate test-only simulator can import a booking into the PMS MongoDB; it is not a Booking.com, Agoda, or Expedia connection.
Roles	Registration labels do not currently grant module permissions. Actual hotel access is owner/member, with limited owner-only access management.

Use with a test hotel first. Creating a property, importing an API test booking, posting a payment, or completing a Night Audit changes MongoDB data. The example API payload later in this guide is illustrative and must not be sent to a live property without review.

1 Start with a hotel workspace

The operational workflow is property-scoped. Sign in with a backend account, then select an existing hotel or create a new one. The backend checks both the signed-in account and the property ID on operational requests. [R1]

Create a property in the PMS

- Register, then sign in. A new registration becomes a backend **user** account regardless of the role label chosen in the form. [R2]
- Open the hotel switcher at the top of the sidebar and choose **Manage / add properties**, or open the property list. Choose **Add property**. [R3]
- Enter hotel name, email, address, two-letter country code, home currency, and timezone; optional city and phone can follow. Select **Create property**. [R3]
- The backend creates a unique property record and assigns the signed-in account as its **owner**. The new hotel starts without the original demo hotel's room and booking records. [R1, R3]
- Under **Settings > Property**, verify hotel details, logo, business date, and invoice text. Then configure **Room & Rates > Rooms** and **Rates** before accepting reservations. [R4]

Give another person access

The other person must register an account first. On **Your properties**, the owner selects **Access**, enters that existing account's email, and grants membership. Members can open the hotel, but cannot manage its access list. The owner can revoke membership. This is the real access path; the separate **Settings > Users** invitation screen is not yet a persistent invitation system. [R1, R3, R5]

> **For real work, sign in. The public demo workspace can only read the 'demo' hotel's MongoDB data in development; it cannot create a hotel or save a booking. Production demo access requires explicit enabling and should contain non-sensitive data. [R1]**

Minimum configuration before a booking

Configure	Where and why
Room type + physical rooms	Room & Rates > Rooms. Add an active room type and physical room numbers; availability is checked against them.
Rate plan	Room & Rates > Rates. Choose currency, room-type price, dates, and restrictions. Test API bookings require a valid rate plan and room type.
Meal allocation (if applicable)	Settings > Property > Meal Allocation. A meal-inclusive rate plan must link to a matching allocation covering the stay.

2 The booking lifecycle at a glance

The normal path is **set up inventory > create a reservation > confirm > prepare the room > check in > collect and reconcile payment > check out > clean the room > close the business day**. A cancelled or no-show booking branches off before check-in. [R6, R7]

Milestone	What you do in the PMS	Saved result
0-10% Prepare	Create/select hotel; add room type, physical room, rate plan, and any meal allocation.	Sellable inventory and prices exist.
10-30% Quote	Front Desk > + Reservation; enter dates, guest, room type, occupancy, plan, and optional room number.	Quoted rate and availability are checked.
30-45% Reserve	Choose Tentative or Confirmed and select Reserve. Correct any inventory or validation error.	Reservation and guest profile are saved in MongoDB.
45-55% Confirm	Confirm a tentative reservation when accepted. Check Financials > Invoices.	Confirmed status; an issued invoice is created automatically.
55-70% Arrive	Check Front Desk / Arrivals. Assign an available, clean physical room and check in on a valid business date.	Room becomes occupied; reservation becomes Checked-in.
70-85% Settle	Open Financials > Invoices, review the invoice, and record payment; verify balance.	Payment and invoice status are updated.
85-95% Depart	Check out from the reservation once the stay ends and finances are reviewed.	Reservation becomes Checked-out; room becomes dirty.
95-100% Close	Complete housekeeping; run Night Audit checks, reports, and Complete Audit.	Room returns clean; business date advances.

The software does NOT currently require a paid invoice before checkout. Treat settlement as a staff checklist item, not an enforced 100% gate. A Night Audit open balance is a warning that staff can acknowledge. [R6, R7]

Other status paths

- **Tentative > Confirmed > Checked-in > Checked-out** is the normal sequence. A new booking can also start Confirmed or Blocked. [R6]
- **Tentative / Confirmed / Blocked > Cancelled** requires a reason. Review any invoice correction or refund before treating it as finished. [R6, R8]
- **Confirmed > No Show** is available when the guest does not arrive. Overdue arrivals otherwise block Night Audit until resolved or overridden. [R6, R7]

3 Do one manual booking in the PMS

A. Create and confirm

- Open the correct hotel, then **Front Desk**. Select **+ Reservation**. The same saved booking can later be found under **Reservation > Bookings**. [R4, R6]
- Enter check-in and check-out, guest name, phone/email, country, adults/children, room type, and rate plan. Check the quoted nightly rate, total, currency, and any restrictions. A rate plan is needed for quoted API pricing. [R4, R6]
- Pick a physical room if appropriate. The backend checks room-type capacity, overlapping reservations, cross-booked rooms, and sellable inventory. For immediate check-in, the room must also be available and clean. [R6]
- Choose **Tentative** if the request is not yet accepted, or **Confirmed** if accepted. Select **Reserve**. If Tentative, open/edit the saved booking and confirm it before check-in. [R6]
- Look up the new reservation number and verify the guest, dates, status, rate, and assigned room. If Confirmed, find the automatically issued invoice in **Financials > Invoices**. [R6, R8]

B. Arrival and payment

- On the arrival date, use **Front Desk** or **Reservation > Arrivals**. Confirm that the room is available and housekeeping shows Clean; then check in. The backend accepts check-in only from Confirmed and within the stay dates. [R6]
- Open **Financials > Invoices**, review the bill, and use **Record Payment** for cash/card/transfer or another supported method. Check the remaining balance and payment status. [R8]
- During the stay, use Front Desk to inspect or edit allowed reservation details, and Housekeeping to record room work. Corrections to an issued invoice should follow the credit/refund flow instead of silently changing historical charges. [R6, R8]

C. Departure and room turnaround

- Before checkout, verify invoice/payment reconciliation manually. Then check out a Checked-in reservation from the PMS. Checkout releases the assigned room and marks it Dirty. [R6]
- In **Housekeeping > Housekeeping Board**, assign/start/complete cleaning. A clean room is ready for the next arrival; dirty or in-progress rooms can block the housekeeping step of Night Audit. [R7, R9]
- Use **Night Audit** at the end of the business day. It closes the day, not the individual reservation. The full procedure is on page 6. [R7]

4 What 'API booking' means here

Path	What is actually working
PMS reservation API	The frontend calls POST /api/bookings/reservations, then status endpoints such as /confirm, /check-in, and /check-out. These write to MongoDB and require a valid signed-in account plus property scope. [R1, R6]
OTA simulator API	POST /api/integrations/ota-simulator/reservations imports a TEST booking to the same PMS reservation collection when enabled. It is not an OTA webhook integration. [R10]
Channel Manager screens	Channel Manager > Bookings uses browser session state. Full Sync and Pull Future Reservations are local simulations and do not exchange data with providers. [R11]

Test-only OTA import: safe mental model

- A developer enables the simulator; a signed-in account must already have access to the target hotel. The request carries the bearer token and a consistent property ID. A separate x-ota-api-key is required only if configured. [R1, R10]
- The payload identifies provider + external reservation ID, guest, dates, room type, rate plan, occupancy, and quantity. The service quotes the rate plan, verifies inventory, upserts the guest, stores the reservation, and auto-issues an invoice if status is Confirmed. [R10]
- Re-sending the same provider/external reservation ID returns the existing reservation rather than creating a duplicate. The imported booking should appear in the PMS Front Desk / Reservation records, but it does not populate the separate Channel Manager browser-only booking list. [R10, R11]

Illustrative request body - do not send to a real hotel

```
POST /api/integrations/ota-simulator/reservations
Authorization: Bearer YOUR_ACCESS_TOKEN
x-property-id: test-hotel-id
Content-Type: application/json

{
  "property_id": "test-hotel-id", "provider": "agoda",
  "external_reservation_id": "TEST-1001", "status": "confirmed",
  "check_in": "2026-10-01", "check_out": "2026-10-02",
  "guest": {"name":"Test Guest", "email":"test@example.com",
            "phone":"+94770000000", "country":"Sri Lanka"},
  "rooms": [{"room_type_id":"ROOM_TYPE_OBJECT_ID",
             "rate_plan_id":"RATE_PLAN_OBJECT_ID",
             "adults":2, "children":0, "quantity":1}]
}
```

This endpoint creates real test records in MongoDB, so use a dedicated test property and real ObjectIds from that property's room and rate setup. It is not automatically called by Booking.com, Agoda, or Expedia. [R10]

5 Night Audit: close the day safely

Open **Night Audit** for the selected hotel. The first access can initialize that property's business date and an open audit in MongoDB. Review the displayed blockers/warnings, then work through the steps below. Completing the audit advances the business date by one day. [R7]

Order	PMS action	What blocks or needs attention
1	Review Front Desk Status	Overdue arrivals and un-checked-out departures are blockers; today's arrivals are warnings.
2	Post Room Revenue	Posts the occupied-room revenue batch. Do not run again as a substitute for invoice/payment reconciliation.
3	Reconcile Payments	Review deposits and open balances. Outstanding balances are warnings, not automatic payment collection.
4	Review Housekeeping Board	Dirty or in-progress rooms block review. Finish room work first.
5	Channel Manager Check	Currently disabled/not required because live provider reconciliation is not connected.
6	Generate Audit Reports	Generates six stored reports: business analysis, deposit ledger, occupancy by date, revenue, inventory by room type, and reservation list.
7	Save optional close note; Complete Audit	Completion requires all required steps done or reviewed with warnings. It closes the day and opens the next business date.

Do not use Manager Override casually. The UI asks for a reason and the API records an approver, but it does not currently enforce that the actor has a Manager role. This is a security/approval gap. [R7]

Audit pack detail: the six reports are generated and stored; the top-level **Download Pack** button currently downloads a JSON snapshot of the audit, not a combined PDF report bundle. [R7]

Operational distinction: checkout is for a guest's reservation; Night Audit is for the hotel's business day. You may close a business day while some guests remain in-house, provided the required checks pass. [R7]

6 Registration roles: intended vs actual

The registration form displays six role choices. Today, the backend ignores the submitted choice and always creates the account with the global role **user**. Those six labels do not grant or restrict module access. [R2]

Selection shown	Likely intended responsibility	Actual effect today
Owner / Administrator	Full hotel configuration, staff, financials, approvals.	No role grant from registration. You become owner only by creating a property or through legacy unowned-hotel admin rules.
Manager	Supervise operations, exceptions, approvals.	No Manager permission or override enforcement.
Front Desk	Reservations, arrivals, departures, guest service.	No Front Desk-only restriction.
Housekeeping	Room assignment, cleaning, status updates.	No Housekeeping-only restriction.
Finance	Invoices, payments, credits, refunds, reports.	No Finance-only restriction.
Reports	View dashboards and reports, ideally read-only.	No Reports-only/read-only restriction.

What access really works

- **Account role:** new public registrations are user. The backend schema also supports admin and developer, but those are not obtainable by selecting a registration label. [R2]
- **Property role:** the creator is owner. An owner can grant/revoke member access to an existing account. Members can use the hotel's operational APIs, and only owner can manage the member list. [R1]
- **Module permission:** there is no enforced per-module role matrix yet. A member with hotel access is not currently prevented from operating Finance, Night Audit, or Settings based on the six registration labels. [R1, R2, R5]
- **Settings > Users:** Add User only updates local React state; it does not create a backend account, send a real invitation, or persist the chosen permission checkboxes. Use Your properties > Access for real membership. [R5]

7 What is not end-to-end complete

Priority	Gap / consequence	Needed before relying on it
Critical	The six registration role labels do not create those roles; hotel members have broad operational access. Night Audit 'Manager Override' has no Manager authorization check.	Design backend-enforced hotel roles and permissions; apply them to every API route and the override action; then reflect them in the UI.
High	No live provider API for OTA bookings, availability/rate sync, acknowledgements, or future-reservation pulls. Channel Manager booking UI is session data.	Implement provider credentials, mapping, webhooks/polling, idempotency, reconciliation, error retries, and a live status monitor.
High	Checkout does not require a settled invoice. Staff can finish a stay with money still due.	Define a credit policy and add a server-side balance/authorization rule if payment must be mandatory.
High	Settings > Users invitations/permission checkboxes are not persisted or enforced.	Connect the page to staff membership/invitation APIs and a real permission model.
Medium	Night Audit posts a room-revenue transaction separately from the issued invoice transaction. Financial reporting should be reconciled for double-counting or accounting-policy differences before live use.	Run an accounting scenario test for invoice, payment, night-audit posting, credits, and reporting across one stay.
Medium	Demo mode is read-only; simulated channel data is not the same as live MongoDB data. Some screens may have locally generated sample values.	Use a signed-in test property for end-to-end testing; label demos clearly and remove sample fallbacks from production flows.

Practical readiness rule: do not describe this as a fully automated OTA-to-checkout workflow yet. The core PMS reservation, invoice, payment, room, and Night Audit paths exist; live channel integration and granular role security still need implementation.

A safe first end-to-end test

- Create a dedicated test hotel and one test staff account; confirm owner/member behavior.
- Add one room type, one clean physical room, one Room Only rate plan, then create a Tentative booking and confirm it.
- Verify the issued invoice; post a small test payment; check in on the correct business date; check out; clean the room.
- Review the Night Audit snapshot, address blockers, generate reports, and close only the test hotel's date. Check report totals and audit history.
- Separately import one OTA simulator booking with a unique external ID; repeat it to verify duplicate protection. Do not expect it in Channel Manager's session-only booking list.

8 Code evidence and scope

This guide is based on the code present in the two local projects on 24 Sep 2026. It is an implementation review, not a live certification of a production deployment. Frontend root: D:\DMS new\new-pms. Backend root: D:\DMS new\hotelbackend.

Ref	Source files and main evidence
R1	Backend services/property-access.service.js; routes/property.js; frontend app/properties/page.tsx, app/lib/property-workspaces.ts. JWT/property guard and owner/member access.
R2	Frontend app/login/page.tsx (six selector options); backend routes/auth.js, db_models/user.model.js (registration always stores role=user).
R3	Frontend app/components/property-switcher.tsx, app/properties/page.tsx; backend routes/property.js (create property, owner assignment, member grants).
R4	Frontend app/data/pms-data.ts (navigation); app/components/modules/rooms-rates/rooms/rooms-page.tsx; rates/rates-page.tsx; settings/property/property-page.tsx.
R5	Frontend app/components/modules/settings/users.tsx (in-memory users and permissions, no API write).
R6	Frontend app/components/modules/front-desk/use-reservation-actions.ts, front-desk/components/reservation-editor.tsx, app/lib/bookings-api.ts; backend routes/bookings.js, services/booking-availability.service.js.
R7	Frontend app/components/modules/nightaudit/nightaudit.tsx; backend routes/night-audit.js, services/night-audit.service.js.
R8	Backend services/invoice-automation.service.js, routes/invoices.js; frontend app/components/modules/financials/financial-documents-page.tsx.
R9	Frontend app/components/modules/housekeeping/board/housekeeping-board-page.tsx; backend housekeeping routes and booking checkout behavior.
R10	Backend routes/ota-simulator.js, services/ota-simulator.service.js; app.js guard configuration.
R11	Frontend app/components/modules/channelmanager/bookings.tsx, fullsync.tsx, pullfuturereservations.tsx, dashboard.tsx.

Terminology: 'Working' means a code path exists and is connected to the backend; it does not mean every production dependency or edge case has been field-tested. 'Local simulation' means browser/session state or preview logic, not a provider API. A staff SOP should be validated with the hotel's actual accounting and privacy policies.